

Commercial Lead Qualification Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Replace the current profile-only lead scorer with a versioned evidence acquisition, semantic extraction, deterministic qualification, and exception-only review flow that matches the approved commercial lead qualification specification.

Architecture: Keep the existing Instagram backfill as the profile acquisition entry point, then run one shared qualification orchestrator for every lead. The orchestrator creates an immutable evidence snapshot, acquires bounded external and YouTube evidence, asks AI only to extract cited facts, and applies deterministic track, eligibility, certainty, scoring, and routing rules. Run the new system in shadow mode first, compare it with reviewer labels, then switch handover eligibility from universal manual approval to explicit auto-approval or human approval.

Tech Stack: Next.js 15, TypeScript, Supabase Postgres, Inngest, Zod, Cheerio, Google YouTube Data API, OpenAI or configured LLM providers, Node test runner through `tsx --test`.

Global Constraints

- Preserve current user changes in the dirty worktree and keep each commit scoped to files touched by this plan.
- Use the approved specification at [docs/superpowers/specs/2026-07-31-commercial-lead-qualification-design.md](#) as the source of truth.
- Do not copy logic from [ATAQ-Lead-Scoring-Spec.pdf](#) into this implementation.
- Never treat missing, failed, unavailable, or not-attempted evidence as evidence of absence.
- Keep activity, followers, likes, comments, views, and posting cadence out of commercial eligibility. Use them only for priority.
- Apply [primary_offer_done_for_you_service](#) before any weighted score and reject it regardless of score, except when a separate independent information funnel is explicitly evidenced.
- Keep AI responsible for semantic evidence extraction only. Track classification, hard exclusions, core gate, scores, certainty, and final outcome remain deterministic TypeScript.
- Preserve old scoring columns during shadow mode so current dashboards continue to work.
- Every new decision must reference an evidence snapshot, extractor version, prompt version, scorecard version, and config version.
- Add unit tests before implementation for every pure rule. Use fixture-based integration tests for acquisition and orchestration.

Current System Gap Summary

Area	Current implementation	Required change
Qualification input	Bio, captions, counts, external URL string	Immutable multi-surface evidence snapshot

Area	Current implementation	Required change
External research	<code>lib/funnel/enrich.ts</code> runs mainly after qualification and selects one best link	Move before qualification, inventory multiple relevant destinations, preserve CTA chain
YouTube	<code>lib/youtube/collect.ts</code> is report-only and discards descriptions and outbound URLs	Use in qualification when relevant and retain descriptions, links, and selection reasons
AI role	Provider returns business model, ICP signal, and offer confidence	Provider returns cited semantic facts, signals, offers, proof, conflicts, and unknowns
Agency handling	Agency maps to partnership and receives monetization credit	Hard reject primary done-for-you service from this ICP
Activity	Low engagement or low recent posting can reject an infopreneur	Remove from eligibility and keep only in priority
Entry points	<code>score-lead.ts</code> and <code>process-profile.ts</code> duplicate qualification logic	Route both through one orchestrator
Rejected records	Score fields are often cleared	Preserve evidence, rule results, scores, and reason codes
Manual review	Every qualified lead needs approval before handover	Auto-approve high-certainty clear cases, review exceptions only
Versioning	No evidence, extraction, or scorecard record chain	Add immutable versioned records and active config
Validation	Almost no tests around scoring or acquisition	Add rule, fixture, orchestration, migration, and benchmark tests

Task 1: Add the test harness and canonical domain contracts

Files:

- Modify: `package.json`
- Create: `lib/qualification/types.ts`
- Create: `lib/qualification/schemas.ts`
- Test: `lib/qualification/schemas.test.ts`
- [] **Step 1: Add a qualification test command**

Add `test:qualification` to run `tsx --test lib/qualification/*.test.ts lib/evidence/*.test.ts lib/funnel/*.test.ts lib/youtube/*.test.ts`.

- [] **Step 2: Write failing schema tests**

Cover capture states, cited evidence, offer inventory, proof attribution, CTA hops, extracted signal states, decision outcomes, and version references. Assert that `captured` with an empty collection is valid and that unknown capture states cannot contain fabricated absence claims.

- [] **Step 3: Define canonical TypeScript contracts**

Create explicit types for `CaptureStatus`, `EvidenceCitation`, `EvidenceSnapshotInput`, `ExternalDestination`, `YouTubeChannelEvidence`, `YouTubeVideoEvidence`, `CtaHop`, `OfferEvidence`, `ProofEvidence`, `CommercialExtraction`, `CommercialDecision`, `DecisionReasonCode`, and `QualificationVersions`.

- [] **Step 4: Add strict Zod validation**

Reject unknown enum members, uncited affirmative facts, invalid CTA hop order, proof without beneficiary attribution state, and decision records without version IDs.

- **[] Step 5: Run the focused test**

Run: `npx tsx --test lib/qualification/schemas.test.ts`

Expected: PASS.

- **[] Step 6: Commit**

```
BASH
git add package.json lib/qualification/types.ts lib/qualification/schemas.ts lib/qualification/
schemas.test.ts
git commit -m "test: define qualification evidence contracts"
```

Task 2: Add immutable evidence, extraction, decision, and config storage

Files:

- Create: `supabase/migrations/20260731010000_commercial_qualification.sql`
- Modify: `lib/types.ts`
- Create: `lib/qualification/repository.ts`
- Test: `lib/qualification/repository.test.ts`

- **[] Step 1: Write repository contract tests with a fake Supabase adapter**

Assert one evidence snapshot can have multiple extraction attempts and decisions, active rows are queryable by lead, and historical records are never updated in place.

- **[] Step 2: Add database tables**

Create `lead_evidence_snapshots`, `lead_commercial_extractions`, `lead_qualification_decisions`, and `lead_qualification_configs`. Include `lead_id`, JSON payload, acquisition status, stop reason, source timestamps, model and provider, token and cost metadata, versions, decision mode, outcome, reason codes, score components, certainty, challenger result, and timestamps.

- **[] Step 3: Add operational columns to leads**

Add `qualification_state`, `qualification_outcome`, `qualification_decision_id`, `qualification_ready_at`, `qualification_review_reason`, and `qualification_pipeline_version`. Keep existing score and review columns intact for compatibility during rollout.

- **[] Step 4: Add indexes and constraints**

Index current lead state, review exceptions, enrichment-ready leads, snapshot lookup, and config activation. Constrain outcome and capture-state values. Do not make old leads invalid.

- **[] Step 5: Implement repository methods**

Provide `createEvidenceSnapshot`, `createExtraction`, `createDecision`, `getLatestQualificationBundle`, `getActiveQualificationConfig`, and `setLeadQualificationProjection`. Inserts create new immutable rows, while only the lead projection is updated.

- **[] Step 6: Regenerate or manually extend application types**

Extend `lib/types.ts` with the new lead projection fields and record types without removing `ClaudeScore` yet.

- **[] Step 7: Verify migration and tests**

Run: `npx supabase db reset`

Run: `npx tsx --test lib/qualification/repository.test.ts`

Expected: migration applies and tests pass.

- **[] Step 8: Commit**

```
BASH
git add supabase/migrations/20260731010000_commercial_qualification.sql lib/types.ts lib/
qualification/repository.ts lib/qualification/repository.test.ts
git commit -m "feat: store versioned qualification evidence"
```

Task 3: Normalize Instagram qualification evidence and data sufficiency

Files:

- Create: `lib/evidence/instagram.ts`
- Create: `lib/evidence/sufficiency.ts`
- Modify: `lib/apify/actors.ts`
- Modify: `lib/instagram/direct.ts`
- Modify: `lib/types.ts`
- Test: `lib/evidence/instagram.test.ts`
- Test: `lib/evidence/sufficiency.test.ts`
- **[] Step 1: Add fixtures for complete, empty, private, failed, and partial profiles**

Include profile category, metadata description, 12 to 18 posts when available, pinned status, and Story Highlight titles. Keep fixture provenance explicit.

- **[] Step 2: Write failing normalization tests**

Assert provider values map into the same normalized Instagram evidence shape and every optional surface has a separate capture state.

- **[] Step 3: Extend provider mapping where data exists**

Capture profile category, Instagram metadata description, pinned post markers, and Highlight titles from Apify or direct responses when returned. When unavailable, persist `unavailable` or `not_attempted`, never an empty captured value.

- **[] Step 4: Implement sufficiency rules**

Return `sufficient`, `retryable`, or `review_required` with reason codes. Require a reliable public profile, bio or metadata fallback, and enough CTA destination state to make a commercial decision. Private, blocked, or provider failure routes to data-quality handling rather than commercial rejection.

- **[] Step 5: Run tests**

Run: `npx tsx --test lib/evidence/instagram.test.ts lib/evidence/sufficiency.test.ts`

Expected: PASS.

- [] Step 6: Commit

```
BASH
git add lib/evidence/instagram.ts lib/evidence/sufficiency.ts lib/evidence/*.test.ts lib/apify/actors.ts lib/instagram/direct.ts lib/types.ts
git commit -m "feat: normalize Instagram qualification evidence"
```

Task 4: Turn funnel enrichment into a bounded commercial evidence collector

Files:

- Modify: `lib/funnel/drill.ts`
- Modify: `lib/funnel/fetch.ts`
- Modify: `lib/funnel/extract.ts`
- Refactor: `lib/funnel/enrich.ts`
- Create: `lib/evidence/external.ts`
- Test: `lib/funnel/drill.test.ts`
- Test: `lib/evidence/external.test.ts`
- Create fixtures: `lib/evidence/fixtures/link-hub.html`,
`lib/evidence/fixtures/coaching-page.html`, `lib/evidence/fixtures/agency-page.html`,
`lib/evidence/fixtures/application-page.html`

- [] Step 1: Write failing link selection and traversal tests

Cover link hubs with coaching, free training, application, YouTube, agency, shop, and social links. Assert the collector keeps all commercially relevant candidates, records selection reasons, follows at most 3 hops by default and 5 absolutely, detects cycles, and reports its stop reason.

- [] Step 2: Refactor selection from one URL to ranked candidates

Replace the single `pickBestFunnelLink` dependency in qualification with `rankFunnelLinks`. Preserve `pickBestFunnelLink` as a compatibility wrapper for reports until downstream callers migrate.

- [] Step 3: Expand deterministic page extraction

Return canonical URL, title, headings, CTA labels and URLs, offer copy, program names, prices, visitor outcomes, destination type, and a bounded text excerpt. Add explicit `agency_service`, `application`, `booking`, `lead_magnet`, `education`, `youtube`, `community`, `store`, and `unknown` classification.

- [] Step 4: Implement the external collector

Use free fetch first and ScrapingBee fallback. Traverse ranked commercially relevant links, store every inspected destination, construct the ordered CTA chain, and return an offer inventory seed. Do not write qualification conclusions here.

- [] Step 5: Separate collection from persistence

Make the new collector pure with respect to lead rows. Keep `enrichFunnelForLead` as a report compatibility adapter that calls the collector and projects legacy funnel columns.

- [] Step 6: Run tests

Run: `npx tsx --test lib/funnel/drill.test.ts lib/evidence/external.test.ts`

Expected: PASS.

- **[] Step 7: Commit**

```
BASH
git add lib/funnel lib/evidence/external.ts lib/evidence/external.test.ts lib/evidence/fixtures
git commit -m "feat: collect bounded external commercial evidence"
```

Task 5: Extend YouTube acquisition for qualification

Files:

- Modify: `lib/youtube/collect.ts`
 - Create: `lib/evidence/youtube.ts`
 - Test: `lib/youtube/collect.test.ts`
 - Test: `lib/evidence/youtube.test.ts`
- **[] Step 1: Write failing YouTube fixture tests**

Cover channel URLs, handles, video links, `youtu.be` links, missing API configuration, primary YouTube CTAs, and unrelated YouTube links.

- **[] Step 2: Preserve qualification evidence**

Return channel description, recent video titles, selected full descriptions, publication times, outbound URLs, and selection reasons. Keep current subscriber, upload cadence, and price fields for report callers.

- **[] Step 3: Add bounded video selection**

Inspect recent videos plus videos whose titles match the profile transformation, offer, free training, application, booking, program, blueprint, roadmap, or method. Store why each video was selected.

- **[] Step 4: Integrate outbound links into the CTA chain**

When YouTube is the primary Instagram CTA, require description capture or a clear capture failure before high certainty is possible. Feed relevant outbound URLs back into the external collector without exceeding the global hop budget.

- **[] Step 5: Run tests**

Run: `npx tsx --test lib/youtube/collect.test.ts lib/evidence/youtube.test.ts`

Expected: PASS.

- **[] Step 6: Commit**

```
BASH
git add lib/youtube/collect.ts lib/youtube/collect.test.ts lib/evidence/youtube.ts lib/evidence/
youtube.test.ts
git commit -m "feat: capture YouTube qualification evidence"
```

Task 6: Build the deterministic evidence snapshot collector

Files:

- Create: `lib/evidence/collect.ts`
- Create: `lib/evidence/versions.ts`
- Test: `lib/evidence/collect.test.ts`

- [] **Step 1: Write orchestration tests with fake collectors**

Assert Instagram evidence is collected first, external and YouTube evidence are conditionally collected, a single global hop budget is enforced, failures are retained, and one immutable snapshot is persisted before AI extraction.

- [] **Step 2: Implement collector orchestration**

Create `collectCommercialEvidence({ lead, settings, dependencies })`. Normalize Instagram evidence, resolve external redirects, collect link-hub children, collect YouTube only when linked or CTA-relevant, merge CTA hops, and generate offer and proof seed inventories.

- [] **Step 3: Add acquisition sufficiency and stop reasons**

Record `complete`, `budget_exhausted`, `cycle_detected`, `blocked`, `provider_failed`, `no_external_link`, or equivalent approved enum values. Calculate whether high-certainty decisioning has enough acquisition coverage.

- [] **Step 4: Add an acquisition version constant**

Store a semantic version and fixture revision in every snapshot. Changing traversal or extraction behavior must require a version increment.

- [] **Step 5: Run tests**

Run: `npx tsx --test lib/evidence/collect.test.ts`

Expected: PASS.

- [] **Step 6: Commit**

```
BASH
git add lib/evidence/collect.ts lib/evidence/versions.ts lib/evidence/collect.test.ts
git commit -m "feat: create immutable commercial evidence snapshots"
```

Task 7: Replace AI fit judgment with cited semantic extraction

Files:

- Create: `lib/qualification/prompt.ts`
- Create: `lib/qualification/extract.ts`
- Create: `lib/qualification/providers.ts`
- Modify: `lib/openai/classify.ts`
- Modify: `lib/claude/classify.ts`
- Modify: `lib/gemini/classify.ts`
- Modify: `lib/groq/classify.ts`
- Test: `lib/qualification/prompt.test.ts`
- Test: `lib/qualification/extract.test.ts`

- [] **Step 1: Write failing prompt and parser tests**

Assert the prompt asks for human personal-brand identity, information funnel, CTA, transformation, proof, authority, visitor outcome, named mechanisms, all offers, offer prominence, proof beneficiary and producing model, agency-service evidence, conflicts, and unknowns. It must not ask the model for an overall score or final outcome.

- [] **Step 2: Implement a provider-neutral extraction request**

Serialize only the stored snapshot and fixed prompt version. Require every affirmative claim to cite source type, source identifier, and excerpt. Bound excerpts and reject citations that do not exist in the snapshot.

- **[] Step 3: Implement provider adapters**

Normalize structured output across configured providers. Use the current provider setting and model routing, but return `CommercialExtraction` rather than `AiClassification` in the new pipeline.

- **[] Step 4: Validate and repair once**

Parse with Zod, allow one structured repair attempt, then route invalid output to review with `ai_output_invalid`. Never fall back to guessing a business model.

- **[] Step 5: Keep the old classifier operational during shadow mode**

Do not delete `scoreProfileRouted` yet. Mark old prompt paths as legacy and prevent new qualification code from importing them.

- **[] Step 6: Run tests**

Run: `npx tsx --test lib/qualification/prompt.test.ts lib/qualification/extract.test.ts`

Expected: PASS.

- **[] Step 7: Commit**

```
BASH
git add lib/qualification lib/openai/classify.ts lib/claude/classify.ts lib/gemini/classify.ts lib/
groq/classify.ts
git commit -m "feat: extract cited commercial evidence with AI"
```

Task 8: Implement deterministic track, exclusion, gate, score, certainty, and priority rules

Files:

- Create: `lib/qualification/classify-track.ts`
- Create: `lib/qualification/eligibility.ts`
- Create: `lib/qualification/score.ts`
- Create: `lib/qualification/certainty.ts`
- Create: `lib/qualification/priority.ts`
- Create: `lib/qualification/decide.ts`
- Test: `lib/qualification/classify-track.test.ts`
- Test: `lib/qualification/eligibility.test.ts`
- Test: `lib/qualification/score.test.ts`
- Test: `lib/qualification/certainty.test.ts`
- Test: `lib/qualification/decide.test.ts`

- **[] Step 1: Encode the approved profile examples as regression fixtures**

Include clear personal-brand information sellers, proof plus CTA cases, weak-engagement but qualified cases, primary done-for-you agencies, mixed agency plus education cases, missing data, and conflicting destinations. Expected results come only from the approved specification and conversation decisions.

- **[] Step 2: Implement deterministic track classification**

Classify `personal_brand_information`, `primary_done_for_you_service`, `mixed_offers`, `non_personal_brand`, or `unknown`. Treat prominence, CTA destination, visitor outcome, and offer delivery as stronger than the words coach or agency alone.

- **[] Step 3: Implement the hard business-model gate**

Reject primary done-for-you delivery before scoring. Allow the independent information-funnel exception only when it has its own audience, information outcome, CTA path, and commercial prominence. Route unresolved mixed offers to targeted review.

- **[] Step 4: Implement the core gate**

Require human personal brand, information funnel, CTA, and at least one of transformation, proof, or authority. Represent every signal as `present`, `absent`, `unknown`, or `conflicting`. Unknown core signals route to retry or review rather than rejection.

- **[] Step 5: Implement the commercial-fit score**

Use the specification's score components and thresholds. Scores explain and rank eligible cases but never override a hard exclusion or failed core gate. Store component-level reasons and citations.

- **[] Step 6: Implement certainty**

High certainty requires sufficient acquisition, a resolved CTA chain, YouTube descriptions when YouTube is primary, offer comparison, proof attribution, no material conflict, valid AI output, and stable deterministic rules.

- **[] Step 7: Implement priority separately**

Use follower scale, recent activity, reach, views, likes, comments, engagement, and commercial strength only after qualification. Missing social metrics lower confidence in priority, not eligibility.

- **[] Step 8: Implement final decision routing**

Return `qualified`, `review`, `rejected`, or `data_retry`, plus `auto_approved`, `manual_review`, `hard_excluded`, or `retry_required`. Preserve scores for all outcomes.

- **[] Step 9: Run all rule tests**

Run: `npm run test:qualification`

Expected: PASS.

- **[] Step 10: Commit**

```
BASH
git add lib/qualification
git commit -m "feat: decide commercial lead eligibility deterministically"
```

Task 9: Add challenger verification for high-impact decisions

Files:

- Create: `lib/qualification/challenger.ts`
- Modify: `lib/qualification/decide.ts`
- Test: `lib/qualification/challenger.test.ts`

- **[] Step 1: Write failing challenger routing tests**

Assert the challenger runs for proposed auto-approvals, mixed offers, conflicts, and configured audit samples. It does not run for reliable universal exclusions or data failures.

- **[] Step 2: Implement a narrow challenger prompt**

Ask a second model or model pass to identify unsupported citations, missed done-for-you service evidence, unresolved CTA outcomes, incorrect proof attribution, and contradictions. Do not ask for a replacement score.

- **[] Step 3: Make challenger disagreement deterministic**

Material disagreement routes to review and records reason codes. Agreement permits the original decision. Store provider, model, prompt version, and result.

- **[] Step 4: Run tests**

Run: `npm test lib/qualification/challenger.test.ts`

Expected: PASS.

- **[] Step 5: Commit**

```
BASH
git add lib/qualification/challenger.ts lib/qualification/challenger.test.ts lib/qualification/decide.ts
git commit -m "feat: challenge high-impact qualification decisions"
```

Task 10: Build one shared Inngest qualification orchestrator

Files:

- Create: `inngest/functions/qualify-lead.ts`
- Create: `lib/qualification/run.ts`
- Modify: `inngest/client.ts`
- Modify: `app/api/inngest/route.ts`
- Modify: `inngest/functions/backfill-metadata.ts`
- Modify: `inngest/functions/score-lead.ts`
- Modify: `inngest/functions/process-profile.ts`
- Test: `lib/qualification/run.test.ts`

- **[] Step 1: Write failing end-to-end orchestration tests with fakes**

Cover complete qualification, retryable acquisition, hard agency exclusion, high-certainty auto-approval, challenger disagreement, and an idempotent rerun. Assert snapshot is stored before extraction and decision is stored before lead projection.

- **[] Step 2: Implement `runCommercialQualification`**

Call sufficiency, collection, extraction, deterministic decisioning, challenger, repository persistence, and projection in one shared service with injected dependencies for tests.

- **[] Step 3: Add the Inngest event and function**

Add `lead/qualify.requested` with `lead_id`, `crawl_job_id`, `force`, `mode`, and optional config version. Use per-lead concurrency and idempotency so duplicate backfill events do not create competing active decisions.

- **[] Step 4: Change backfill fan-out**

Send `lead/qualify.requested` after metadata backfill instead of directly scoring when the new pipeline is enabled. Preserve the old event in shadow and rollback modes.

- **[] Step 5: Remove duplicated decision logic from active entry points**

Make `process-profile.ts` persist profile data and dispatch the shared qualification event. Make `score-lead.ts` a compatibility or legacy-shadow wrapper during rollout. There must be one production path that decides commercial eligibility.

- **[] Step 6: Register the function and verify replay behavior**

Register `qualifyLead` in `app/api/ingest/route.ts`. Test repeated events, partial external failures, cancellation, and force reprocessing.

- **[] Step 7: Run tests and typecheck**

Run: `npx tsx --test lib/qualification/run.test.ts`

Run: `npm run typecheck`

Expected: PASS.

- **[] Step 8: Commit**

```
BASH
git add ingest/functions/qualify-lead.ts ingest/functions/backfill-metadata.ts ingest/functions/score-lead.ts ingest/functions/process-profile.ts ingest/client.ts app/api/ingest/route.ts lib/qualification/run.ts lib/qualification/run.test.ts
git commit -m "feat: orchestrate evidence-first lead qualification"
```

Task 11: Replace universal manual review with exception-only review

Files:

- Create: `supabase/migrations/20260731020000_qualification_review_routing.sql`
- Modify: `app/actions/review.ts`
- Modify: `lib/handover/batch.ts`
- Modify: `lib/handover/overview.ts`
- Modify: `app/(dashboard)/review/page.tsx`
- Modify: `components/review/review-client.tsx`
- Modify: `app/(dashboard)/leads/[username]/page.tsx`
- Test: `lib/qualification/review-routing.test.ts`

- **[] Step 1: Write failing routing tests**

Assert auto-approved qualified leads are immediately handover eligible, human-approved review cases become eligible, rejected and data-retry cases never become eligible, and deferred review remains non-final.

- **[] Step 2: Add explicit approval source**

Persist `approval_source` as `automatic` or `human`, plus decision ID and timestamp. Backfill existing human-approved rows without changing their eligibility.

- **[] Step 3: Change handover queries**

Replace the strict `review_decision = approved` dependency with an explicit `qualification_ready_at is not null` or equivalent approved projection. Keep a temporary compatibility condition for pre-migration rows.

- **[] Step 4: Make review queue exception-specific**

Query only `qualification_outcome = review`. Show the exact missing, conflicting, acquisition, challenger, or mixed-offer reason and the supporting citations. Do not ask reviewers to re-research clear auto-approved leads.

- **[] Step 5: Add decision evidence to the lead detail page**

Display funnel type, CTA chain, transformation, proof, authority, offer inventory, agency exclusion result, certainty, source links, capture failures, and versions.

- **[] Step 6: Run tests and typecheck**

Run: `npx tsx --test lib/qualification/review-routing.test.ts`

Run: `npm run typecheck`

Expected: PASS.

- **[] Step 7: Commit**

```
BASH
git add supabase/migrations/20260731020000_qualification_review_routing.sql app/actions/review.ts
lib/handover/batch.ts lib/handover/overview.ts app/\(dashboard\)\/review/page.tsx components/review/
review-client.tsx app/\(dashboard\)\/leads\/\[username\)\/page.tsx lib/qualification/review-routing.
test.ts
git commit -m "feat: route only qualification exceptions to review"
```

Task 12: Add shadow mode, reprocessing, observability, and release controls

Files:

- Create: `supabase/migrations/20260731030000_qualification_rollout.sql`
- Modify: `app/actions/settings.ts`
- Modify: `components/settings/settings-form.tsx`
- Create: `app/actions/qualification.ts`
- Create: `lib/qualification/metrics.ts`
- Create: `scripts/reprocess-commercial-qualification.ts`
- Modify: `components/logs/pipeline-stats.tsx`
- Modify: `components/dashboard/pipeline-flow.tsx`
- Test: `lib/qualification/metrics.test.ts`

- **[] Step 1: Add rollout settings**

Support `legacy`, `shadow`, `review_only`, and `active` modes, active config version, challenger percentage, blind audit percentage, acquisition budgets, and provider/model selection. Default the migration to `shadow`.

- **[] Step 2: Add safe historical reprocessing**

Create a script and server action that enqueue IDs in bounded batches, skip already-current snapshots unless forced, and never overwrite historical records. Include filters for raw backfilled, legacy qualified, legacy rejected, and reviewer-labeled leads.

- **[] Step 3: Add decision and funnel metrics**

Measure acquisition completion, capture failures by source, external hop distribution, AI validation failures, challenger disagreements, hard agency exclusions, core-gate failures, auto-approval rate, manual-review rate, reviewer overturn rate, cost per lead, latency, and enrichment-ready throughput.

- **[] Step 4: Add blind audits**

Sample a configurable percentage of auto-approved and rejected decisions into a separate audit queue without blocking enrichment. Record reviewer agreement independently from operational review.

- **[] Step 5: Update pipeline UI labels**

Show [Backfilled](#), [Evidence collecting](#), [AI extracting](#), [Decisioning](#), [Auto-approved](#), [Needs review](#), [Rejected](#), [Data retry](#), and [Enrichment-ready](#). Keep old counts visible during shadow comparison.

- **[] Step 6: Run tests and typecheck**

Run: `npx tsx --test lib/qualification/metrics.test.ts`

Run: `npm run typecheck`

Expected: PASS.

- **[] Step 7: Commit**

```
BASH
git add supabase/migrations/20260731030000_qualification_rollout.sql app/actions/settings.ts
components/settings/settings-form.tsx app/actions/qualification.ts lib/qualification/metrics.ts lib/
qualification/metrics.test.ts scripts/reprocess-commercial-qualification.ts components/logs/
pipeline-stats.tsx components/dashboard/pipeline-flow.tsx
git commit -m "feat: control and measure qualification rollout"
```

Task 13: Benchmark, cut over, and retire legacy rejection logic

Files:

- Create: `lib/qualification/fixtures/benchmark.json`
- Create: `scripts/benchmark-commercial-qualification.ts`
- Create: `docs/qualification-rollout.md`
- Modify: `lib/pipeline/filter.ts`
- Modify: `lib/scoring/compute.ts`
- Modify: `lib/scoring/score.ts`
- Modify: `lib/leads/category.ts`
- Modify: `ingest/functions/score-lead.ts`
- Modify: `ingest/functions/process-profile.ts`

- **[] Step 1: Build a labeled benchmark set**

Include the user's known very-qualified profiles, agency false positives such as the reviewed agency case, proof plus CTA examples, low-engagement qualified leads, mixed offers, and current reviewer approvals and rejections. Store evidence snapshots or sanitized fixtures, not live-only URLs.

- **[] Step 2: Define release thresholds**

Require zero primary done-for-you agency false positives in the agency fixture set, at least 95 percent precision on auto-approved leads, at least 90 percent recall on known qualified leads, less than 15 percent manual-review routing

on sufficient evidence, and no statistically meaningful handover regression. Adjust thresholds only through a documented config change.

- **[] Step 3: Run shadow comparison**

Run the new system on a representative historical sample. Compare legacy outcome, new outcome, reviewer label, reason codes, acquisition completeness, model cost, and latency. Investigate every auto-approved false positive and every known-qualified false negative.

- **[] Step 4: Run review-only mode**

Let new decisions populate the exception queue while legacy decisions still control handover. Validate that reviewers can resolve exceptions from stored evidence without browsing in the normal case.

- **[] Step 5: Enable active mode**

Switch the active config only after thresholds pass. Monitor daily for seven days with blind audits and a documented rollback to legacy or review-only mode.

- **[] Step 6: Remove legacy eligibility effects**

After the observation window, remove follower range, include-keyword, recent-post, engagement, and reel-cadence rejection from the active qualification path. Remove agency monetization credit and agency-to-partnership qualification behavior. Keep reusable metric computation for priority and reporting.

- **[] Step 7: Preserve compatibility or remove dead code explicitly**

Either retain legacy scoring behind `legacy` mode with a removal date or delete it in a dedicated follow-up commit. Do not leave two unversioned active decision paths.

- **[] Step 8: Run full verification**

Run: `npm run test:qualification`

Run: `npm run check`

Run: `npx tsx scripts/benchmark-commercial-qualification.ts`

Expected: all tests and build pass, benchmark thresholds pass, and no placeholder markers remain.

- **[] Step 9: Scan the implementation**

Run: `rg -n "TODO|FIXME|placeholder|coming soon|primary_offer_done_for_you_service|no_recent_posts|low_engagement|low_reel_cadence" lib/qualification lib/evidence ingest/functions app/actions lib/pipeline lib/scoring docs/qualification-rollout.md`

Expected: no unintended placeholders, active done-for-you exclusion is present, and legacy activity rejection appears only in documented legacy code.

- **[] Step 10: Commit**

```
BASH
git add lib/qualification/fixtures/benchmark.json scripts/benchmark-commercial-qualification.ts
docs/qualification-rollout.md lib/pipeline/filter.ts lib/scoring/compute.ts lib/scoring/score.ts
lib/leads/category.ts ingest/functions/score-lead.ts ingest/functions/process-profile.ts
git commit -m "feat: activate evidence-first lead qualification"
```

Recommended Delivery Order

1. Foundation: Tasks 1 to 3.
2. Evidence acquisition: Tasks 4 to 6.
3. Extraction and deterministic decisions: Tasks 7 to 9.
4. Pipeline and review integration: Tasks 10 to 11.
5. Shadow rollout and production cutover: Tasks 12 to 13.

The first production value checkpoint is the end of Task 10, where the full new decision can run in shadow mode. Do not change handover eligibility before shadow benchmarks show that the new system meets the release thresholds.

Final Verification Checklist

- ☐ Every affirmative signal links to stored source evidence.
- ☐ Every missing surface has an explicit capture state.
- ☐ Primary done-for-you services cannot pass through weighted scoring.
- ☐ Proof plus CTA is not enough unless an information funnel, human personal brand, and at least one supporting signal are established.
- ☐ Weak engagement or posting cadence cannot reject a commercially qualified lead.
- ☐ Clear high-certainty leads bypass manual review.
- ☐ Mixed, conflicting, incomplete, and challenger-disputed cases enter targeted review.
- ☐ All qualification entry points use one shared orchestrator.
- ☐ Historical snapshots and decisions remain replayable.
- ☐ Shadow metrics and blind audits make accuracy measurable after release.